Continuous 4D Code Personality

The Style Engine — Deterministic Post-Processing for Code Aesthetics

---

David Jean Charlot, PhD Open Interface Engineering, Inc. (openIE) University of California, Santa Barbara (UCSB) david@openie.dev | dcharlot@ucsb.edu

January 2026 | Version 1.0

Abstract

Large language models spend trillions of parameters learning code style as an implicit side effect of training. When asked to "write Pythonic code" versus "write terse C," the model activates different regions of its parameter space to produce output that *feels* different — different naming conventions, different comment density, different documentation depth. The correctness is the same; the personality is different. This personality consumes billions of parameters and produces probabilistic, non-reproducible results.

We present the **Style Engine**, a deterministic post-processing system that decomposes code style into a continuous 4-dimensional space with explicit transforms. The four axes — voice (comment density), documentation level, naming verbosity, and header weight — form a unit hypercube where any coordinate produces a distinct, reproducible code personality. Eight named presets (Kernighan, Knuth, Torvalds, Dijkstra, Pythonista, Rustacean, Hacker, Clean) are specific coordinates in this space, not the space itself. Custom coordinates, linear interpolation between presets, and gradient-based optimization are first-class operations.

Five deterministic transforms applied in fixed order — strip noise, rename arguments, adjust voice, restyle header, inject documentation — convert raw synthesized code into styled output. Each transform exhibits gradient behavior within its operating band: no flat zones, no discontinuities, continuous variation across the entire axis range. The system operates across 37 target languages via per-language comment syntax tables.

A GPU-accelerated optimizer (Apple Metal) searches 160,000 candidate style coordinates in 9.56ms, identifying the optimal profile for a given style objective. A hybrid optimizer combines GPU grid search with CPU gradient descent for 60.9% loss reduction versus CPU-only optimization. StyleMemory provides per-pattern, per-domain, and global preference learning via exponential moving average, enabling the system to remember and adapt to user preferences over time.

The Style Engine replaces what LLMs capture implicitly across trillions of parameters with approximately 1,800 lines of deterministic code. Same algorithm. Same correctness. Different soul — on demand, in microseconds, reproducibly.

Keywords: code style, code personality, deterministic transforms, GPU optimization, style transfer, preference learning, post-processing

---

1. Introduction

1.1 The Style Problem

Consider two implementations of the Euclidean GCD algorithm. The first is terse — single-character variable names, no comments, no documentation. The second is literate — descriptive names, inline comments explaining each step, a full docstring with parameter types and complexity analysis. Both are correct. Both compute the same result. They differ only in *personality*.

In the LLM paradigm, these two outputs come from different prompts, different sampling paths, different random seeds. The model has no explicit representation of “style” — it is an emergent property of trillions of parameters

tuned across millions of code examples. This has three consequences:

1. **Non-reproducibility**: The same prompt may produce different style on different runs
2. **Non-composability**: There is no way to specify "70% Knuth + 30% Torvalds"
3. **Non-optimizability**: There is no gradient to follow from one style toward another

The Style Engine eliminates all three problems by making style explicit, continuous, and deterministic.

1.2 Decomposition Thesis

Style can be decomposed and applied as deterministic post-processing. A correct algorithm is a correct algorithm regardless of how its variables are named, how many comments surround it, or what header metadata precedes it. These aesthetic properties are orthogonal to correctness and can be applied *after* synthesis as string transformations on the generated code.

This decomposition has a remarkable consequence: the synthesis system (the Code Factory's OODA engagement loop [1]) needs to solve only for *correctness*. Style is a separate, orthogonal concern applied after the fact. This separation of concerns reduces both the synthesis problem and the style problem to their irreducible cores.

1.3 Contributions

This paper describes:

1. **A continuous 4D style space** where every coordinate produces a distinct code personality (Section 2)
2. **Five deterministic transforms** with gradient behavior within operating bands (Section 3)
3. **GPU-accelerated style optimization** via Apple Metal compute kernels (Section 5)

4. **Style transfer** — extracting a style objective from reference code and optimizing new code to match (Section 6)
5. **StyleMemory** — per-pattern, per-domain, and global preference learning (Section 7)

---

2. The 4D Style Space

2.1 Axis Definitions

The style space is a unit hypercube $[0, 1]^4$ with four axes:

Axis	Meaning	Bands
`voice`	Comment density	0.0=silent, 0.15–0.40=terse, 0.40–0.65=balanced, 0.65–0.85=verbose, 0.85–1.0=poetic
`doc_level`	Documentation depth	0.0–0.17=none, 0.17–0.50=minimal, 0.50–0.83=standard, 0.83–1.0=academic
`naming`	Naming verbosity	0.0–0.17=single-char, 0.17–0.50=short, 0.50–0.83=full, 0.83–1.0=mathematical
`header`	Header weight	0.0–0.17=none, 0.17–0.50=minimal, 0.50–0.83=standard, 0.83–1.0=attribution

Each axis is continuous. The band labels describe qualitative behavior at different regions of the axis, but transitions between bands are smooth — there are no discontinuities.

2.2 Named Presets

Eight presets are specific coordinates, named after programming luminaries whose coding style they evoke:

Preset	voice	doc_level	naming	header	Signature
Kernighan	0.25	0.33	0.00	0.33	K&R style
Knuth	0.75	1.00	0.66	1.00	Literate style
Torvalds	0.25	0.00	0.33	0.33	Kernel style
Dijkstra	0.50	0.66	1.00	0.66	Structured programming
Pythonista	0.50	0.66	0.66	0.66	Pythonic
Rustacean	0.50	0.33	0.66	0.33	Idiomatic Rust
Hacker	0.25	0.00	0.00	0.00	(minimal)
Clean	0.50	0.66	0.33	0.66	Clean Code

These presets span the style space but do not exhaust it. Custom coordinates like `StyleProfile::new(0.42, 0.71, 0.55, 0.20)` are equally valid and produce interpolated behavior.

2.3 Interpolation

Linear interpolation between any two profiles is a first-class operation:

$$\text{blend}(A, B, t) = A \cdot (1 - t) + B \cdot t, \quad t \in [0, 1]$$

Applied axis-wise: `blend(Hacker, Knuth, 0.3)` produces a profile with 30% of the distance from Hacker to Knuth on each axis. This enables smooth transitions between coding personalities and "in-between" styles that no single preset captures.

---

3. The Five Transforms

The Style Engine applies five transforms in a fixed order. Order matters: earlier transforms prepare the code for later transforms (e.g., noise stripping must precede header injection).

3.1 Transform 1: Strip Noise

Removes synthesis artifacts from the raw CodeGen output:

- Auto-generation comments (`"Auto-Generated by Cortex CodeGen"`)
- Language-agnostic boilerplate (`"Language-agnostic pattern synthesized"`)
- Rust-specific noise (`#![allow(...)]`, `use cortex_std::prelude::*`)
- Empty comments (`//`, `#`, `--`, `;;`) at low voice settings
- Consecutive blank lines (collapsed to single)
- Leading blank lines

This transform is voice-aware: at silent/terse voice levels, bare comment markers are removed; at balanced and above, they are preserved.

3.2 Transform 2: Rename Arguments

Replaces positional argument names (`arg0`, `arg1`, ...) with semantically meaningful names drawn from pattern metadata. The naming strategy varies continuously across the naming axis:

Band	Range	Strategy	Example: `("array", "i64")`
Single-char	0.00–0.17	First character of semantic name	`a`
Short	0.17–0.50	Abbreviated form (3–4 chars)	`arr`
Full	0.50–0.83	Full name, short names auto-expanded	`array`
Mathematical	0.83–1.00	Greek/math notation	`A`

The full band includes automatic expansion of single/double-character names via a 22-entry mapping table: `a` → `value`, `n` → `count`, `i` → `index`, etc. The mathematical band maps to domain-appropriate symbols: `matrix` → `M`, `learning_rate` → `η`, `gradients` → `∇`.

3.3 Transform 3: Adjust Voice

Controls inline comment density with gradient behavior within each band:

Silent (voice < 0.15): All non-doc comments stripped. Zero comment lines in output.

Terse (0.15 ≤ voice < 0.40): Gradient retention — a fraction of existing comments are retained. The retention rate is:

$$\text{retention} = \frac{\text{voice} - 0.15}{0.25}$$

At voice=0.15: retention=0% (all stripped). At voice=0.40: retention=100% (all kept). Between these bounds, comments are deterministically selected via a hash function: `hash = (comment_index × 7 + 13) mod 100`. A comment is retained if `hash < retention × 100`. This produces reproducible, smoothly varying comment density.

Balanced (0.40 ≤ voice < 0.65): Passthrough. Existing comments preserved as-is.

Verbose (0.65 ≤ voice < 0.85): Structural comments are *added* before code constructs. The verbosity level controls which construct types receive comments:

$$\text{verbosity} = \frac{\text{voice} - 0.65}{0.20}$$

Verbosity	Comment Types Added
0.00	Loops only
≥ 0.25	+ Declarations
≥ 0.33	+ Conditionals
≥ 0.50	+ Assignments
≥ 0.66	+ Returns
≥ 0.75	+ Function calls

Six comment types total — loops, conditionals, returns, declarations, assignments, and calls — enable comment density up to approximately 30% of total lines.

Poetic (voice ≥ 0.85): All six structural comment types are added with poetic text variants: loops become “and so the dance begins…”, conditionals become “a fork in the road”, returns become “the answer reveals itself”. Comment density can reach approximately 35%.

3.4 Transform 4: Restyle Header

Adds metadata header before the code body with continuous depth:

$$\text{depth} = \text{header} \times 6.0$$

Depth	Fields Included
< 1.0	Nothing
1.0–2.0	Pattern ID
2.0–3.0	+ Description (inline)
3.0–4.0	+ Time/space complexity
4.0–5.0	+ Category/subcategory
≥ 5.0	+ Difficulty, block comment wrapper

At depth ≥ 5.0, the header is wrapped in language-appropriate block comment syntax (`/* ... */` for C-family, `""" ... """` for Python, etc.).

3.5 Transform 5: Inject Documentation

Generates documentation blocks (docstrings, doc comments) above the function definition. The depth of documentation varies continuously within bands:

None (doc_level < 0.17): No documentation injected.

Minimal (0.17 ≤ doc_level < 0.50): Single-line doc comment with the pattern description.

Standard (0.50 ≤ doc_level < 0.83): Multi-section documentation with continuous field inclusion:

$$\text{doc\_depth} = \frac{\text{doc\_level} - 0.50}{0.33}$$

doc_depth	Fields
≥ 0.00	Description
≥ 0.33	+ Parameter descriptions with types
≥ 0.66	+ Return type documentation

Academic (doc_level ≥ 0.83): All standard fields plus complexity analysis and category. An additional sub-gradient:

$$\text{academic\_depth} = \frac{\text{doc\_level} - 0.83}{0.17}$$

At academic_depth ≥ 0.5: category and subcategory fields are included.

---

4. Language Support

4.1 Comment Syntax Tables

The Style Engine defines per-language comment syntax for 37 target languages via the `CommentSyntax` struct:

Language Group	Line	Block Start	Block End	Doc Line
C-family (C, C++, Java, C#, Kotlin, Scala, Swift, Dart, JS, TS, Groovy, Zig, D, V)	`//`	`/*`	`*/`	`///`
Rust	`//`	`/*`	`*/`	`///`
Python	`#`	`"""`	`"""`	`#`
Ruby, Perl, Bash, R, Julia, Nim, Elixir, Mojo	`#`	`#`	`#`	`#`
Haskell, Lua	`--`	`--`	`--`	`-- |`
Ada	`--`	`--`	`--`	`--`
OCaml	`(*`	`(*`	`*)`	`(**`
F#	`//`	`(*`	`*)`	`///`
Clojure	`;;`	`;;`	`;;`	`;;`
COBOL	`*>`	`*>`	`*>`	`*>`
Fortran	`!`	`!`	`!`	`!`
Go	`//`	`/*`	`*/`	`//`

All five transforms use these syntax tables, ensuring that comments, headers, and documentation are generated in the correct format for each target language.

4.2 Function Detection

The inject_docs transform must locate the function definition to place documentation above it. The `find_function_line` method uses language-specific heuristics:

- **Python:** Lines starting with `def`
- **Rust:** Lines starting with `fn` or `pub fn`
- **Go:** Lines starting with `func`
- **Java/C#/Kotlin/Scala:** Lines containing `public`, `private`, `static`, or `fun` with parentheses
- **JavaScript/TypeScript:** Lines starting with `function`, `const`, or `export`
- **Haskell:** Lines containing `::` or lowercase-initial with spaces (type signatures)
- **Fallback:** Lines containing `fn`, `func`, `def`, `function` with parentheses

---

5. GPU-Accelerated Style Optimization

5.1 Motivation

Finding the optimal style profile for a given objective requires searching the 4D style space. A brute-force search at resolution R evaluates R^4 candidates — at $R = 8$, that is 4,096 evaluations. Each evaluation requires applying all five transforms and measuring the resulting metrics. On CPU, this is sequential and slow.

5.2 Architecture

The GPU optimizer replaces the full transform pipeline with an analytical model. The key insight: the Metal compute kernel encodes the piecewise-linear behavior of each transform as arithmetic operations on `f32` values. The GPU *predicts* style metrics (comment density, doc coverage, header depth, identifier length) directly from base code features and style coordinates, without performing string manipulation.

```
CPU:  Extract BaseCodeFeatures from unstyled code (once per pattern)
        │
GPU:  Parallel search over R^4 candidates (4096 threads at R=8)
        │
        ├── Grid search: Each thread decodes 4D coordinate, predicts
metrics,
        │                 computes loss against objective
        └── Gradient descent: Finite-difference gradient, in-place
position update
        │
CPU:  Decode winning coordinates, apply actual style transforms
```

5.3 Base Code Features

The `BaseCodeFeatures` struct (`#[repr(C)]` for Metal buffer transfer) captures:

Field	Type	Purpose
`total_lines`	`u32`	Total line count
`comment_lines`	`u32`	Pre-existing comment lines
`blank_lines`	`u32`	Blank line count
`code_lines`	`u32`	Non-blank, non-comment lines
`avg_ident_chars`	`f32`	Average identifier character length
`has_function`	`u32`	Whether a function definition was detected
`arg_count`	`u32`	Number of function arguments
`has_description`	`u32`	Whether pattern description is available

These features are extracted once on CPU and transferred to GPU memory for all subsequent evaluations.

5.4 Two GPU Operations

Grid Search (`style_grid_search`): Dispatches R^4 threads in a 1D grid. Each thread decodes its index into a 4D coordinate (v, d, n, h) via modular arithmetic, predicts all four metrics using the analytical model, and computes the weighted squared loss against the objective. An atomic minimum reduction identifies the best candidate.

Gradient Descent (`style_gradient_step`): Dispatches 1 thread per pattern. Each thread computes finite-difference gradients (perturb each axis by ϵ, evaluate loss, compute $(L(x + \epsilon) - L(x))/\epsilon$) and updates the position in-place. Multiple steps refine the position toward the local minimum.

5.5 Performance

From the GPU style demo (Apple Silicon): - 160,000 candidates (R=20) searched in 9.56ms - 5-pattern batch gradient descent in 4.01ms - Approximately 2.4x faster than CPU gradient computation

5.6 Hybrid Optimizer

The `hybrid_optimize` function combines both approaches:

1. **GPU grid search** at $R = 8$ identifies the global region (4,096 candidates)
2. **CPU gradient descent** refines the position within that region (iterative steps with learning rate and momentum)

On the Hacker→Knuth transfer task (maximally different presets), the hybrid optimizer achieves 60.9% loss reduction versus CPU-only gradient descent.

---

6. Style Transfer

6.1 Objective Extraction

Given a reference code sample, `objective_from_reference(code, lang)` extracts measurable style metrics and constructs a `StyleObjectiveGpu` with normalized weights:

Weight	Value	Metric
`w_comment`	4.0	Comment density (dominant signal)
`w_doc`	1.0	Documentation coverage
`w_header`	0.03	Header depth (discrete, less signal)
`w_naming`	0.04	Identifier length (low discrimination)

Comment density receives the highest weight because it is the most perceptible style dimension — the difference between heavily commented and uncommented code is immediately visible. Header depth and naming receive low weights because they are coarser signals with less discriminating power.

6.2 Transfer Pipeline

```
Reference code → measure() → StyleObjectiveGpu
                                        │
New code → BaseCodeFeatures → GPU grid search → optimal profile
                                                        │
New code → StyleEngine::apply(profile) → styled output
```

The `solve_like(challenge, lang, reference)` method on CodeFactory performs this pipeline end-to-end: solve the challenge, extract style objective from reference, optimize, apply.

6.3 Comment Density Correction

Both CPU and Metal comment density calculations exclude blank lines from the denominator:

$$\text{comment\_density} = \frac{\text{comment\_lines}}{\text{total\_lines} - \text{blank\_lines}}$$

This prevents blank-line-heavy code from artificially deflating the perceived comment density and ensures consistent metrics between the CPU measurement and GPU prediction paths.

---

7. StyleMemory — Preference Learning

7.1 Architecture

StyleMemory maintains three tiers of learned preferences:

Tier	Key	Confidence Threshold	Purpose
Pattern	Pattern ID (e.g., `gcd`, `fibonacci`)	> 0.5	Per-algorithm preferences
Domain	Domain name (e.g., `math`, `networking`)	> 0.3	Per-domain preferences
Global	(singleton)	> 0.2	Fallback preferences

Each tier stores a `StylePreference` with the four style axes, confidence, observation count, and last-updated timestamp.

7.2 Learning Rule

Preferences are updated via exponential moving average (EMA):

$$\text{factor} = \alpha \cdot \text{satisfaction}$$

$$\text{axis} = \text{axis} \cdot (1 - \text{factor}) + \text{input} \cdot \text{factor}$$

Where $\alpha = 0.3$ is the learning rate and satisfaction $\in [0, 1]$ represents user satisfaction with the styled output. High-satisfaction observations shift the preference more strongly; low-satisfaction observations produce minimal updates.

Confidence grows with observations: each observation increases confidence by a fixed increment, capped at 1.0. This prevents the system from making strong recommendations based on insufficient data.

7.3 Suggestion Hierarchy

When queried for a style suggestion, StyleMemory follows a fallback chain:

1. **Pattern-level** (confidence > 0.5): If the specific pattern has been seen enough times, use its learned preference
2. **Domain-level** (confidence > 0.3): If the domain has accumulated enough observations, use the domain preference
3. **Global** (confidence > 0.2): Use the global learned preference
4. **None**: Insufficient data — caller should use a default style

7.4 Persistence

StyleMemory derives `Serialize` and `Deserialize` (via serde + bincode). The `persistence_path` field is `#[serde(skip)]`. Auto-save triggers every 50 observations. The `with_persistence(path)` constructor loads existing state from disk if available.

7.5 Integration

The CodeFactory provides end-to-end integration:

- `solve_remembered(challenge, lang)` — solve with learned style preference
- `record_style_feedback(pattern_id, domain, profile, satisfaction)` — record user feedback
- `style_summary()` — inspect learned preferences
- `style_memory_ref()` — direct access to the StyleMemory instance

The OODA engagement loop's `Engagement::execute()` automatically calls `apply_learned_style()` after synthesis when StyleMemory is enabled. Doctrine recipes also apply learned preferences.

---

8. Styled Code Metrics

8.1 Measurement

The `StyledCodeMetrics` struct captures 8 measurable features of styled code:

Metric	Type	Description
`line_count`	`usize`	Total lines
`comment_count`	`usize`	Non-doc comment lines
`comment_density`	`f64`	comment_count / (line_count - blank_lines)
`doc_lines`	`usize`	Documentation lines
`doc_coverage`	`f64`	Fraction of possible doc fields present (0.0–1.0)
`header_depth`	`usize`	Header metadata fields (0–6)
`avg_identifier_length`	`f64`	Mean length of function argument identifiers
`blank_line_ratio`	`f64`	blank_lines / total_lines

8.2 Gradient Computation

The `compute_gradient` function computes the 4x5 Jacobian matrix of metrics with respect to style axes via forward finite differences:

$$\frac{\partial m_j}{\partial s_i} \approx \frac{m_j(s_i + \epsilon) - m_j(s_i)}{\epsilon}$$

Where s_i are the four style axes and m_j are the five differentiable metrics (comment_density, doc_coverage, header_depth, avg_ident_length, blank_ratio). This Jacobian enables gradient-based optimization toward any target metric vector.

8.3 Loss Function

The style objective defines a weighted squared loss:

$$L = w_c \cdot (c - c^*)^2 + w_d \cdot (d - d^*)^2 + w_h \cdot (h - h^*)^2 + w_n \cdot (n - n^*)^2$$

Where c, d, h, n are current metrics, c^*, d^*, h^*, n^* are targets, and w_c, w_d, w_h, w_n are weights. The CPU optimizer performs gradient descent on this loss using the computed Jacobian.

---

9. Evaluation

9.1 Transform Coverage

The Style Engine has been verified across 8 style presets, 37 target languages, and 800 patterns. The test suite (53 tests) covers:

- All five transforms in isolation and composition
- All eight presets
- Custom profiles and interpolation
- Edge cases (empty code, missing metadata, unknown languages)
- Gradient behavior within each voice band
- StyleMemory recording, suggestion, persistence, and confidence thresholds

9.2 Benchmark Results

The benchmark suite (8 style tests, all passing) verifies:

Test	Description
Style presets produce distinct outputs	Same code, 8 different presets → 8 different outputs
Naming gradient	voice=0.0 through voice=1.0 → increasing identifier length
Comment density gradient	voice=0.0 through voice=1.0 → increasing comment count
Doc coverage gradient	doc=0.0 through doc=1.0 → increasing doc sections
Header depth gradient	header=0.0 through header=1.0 → increasing metadata
Blend continuity	blend(A, B, t) varies smoothly as t increases
StyleMemory learning	Preferences converge with repeated observations
GPU/CPU metric agreement	GPU predicted metrics match CPU measured metrics within tolerance

9.3 Composition with Style

The `composed_style_search` function demonstrates that style applies to composed (multi-pattern) programs identically to single patterns. A staged pipeline solves individual patterns, applies style, and ranks by combined score:

$$\text{combined\_score} = \text{composition\_score} - w_s \cdot \text{style\_loss}$$

This enables "find the best composition that matches this style" — combining functional correctness with aesthetic preference in a single optimization.

---

10. Related Work

10.1 Code Style Transfer

Munson et al. [2] explore code style transfer using pre-trained language models fine-tuned on style-labeled data, defining style attributes for Python and evaluating neural networks' ability to capture and reproduce programming style.

Our approach is entirely deterministic — no training data, no neural networks, no probabilistic sampling. The transform rules encode domain knowledge about what constitutes “Kernighan style” versus “Knuth style” directly as code.

10.2 Code Beautifiers and Formatters

Tools like Prettier [3], Black [4], and rustfmt enforce syntactic formatting rules (indentation, line length, brace placement). The Style Engine operates at a higher level of abstraction: it controls *what* appears in the code (comments, documentation, headers, naming conventions), not *how* the code is formatted. The two are complementary — the Style Engine produces styled code that a formatter can then layout.

10.3 Configurable Code Generation

Template-based code generators (e.g., Yeoman, Cookiecutter) support configuration options for generated code. These are typically boolean flags (include tests: yes/no, include docs: yes/no). The Style Engine’s continuous 4D space provides infinitely fine-grained control — not “docs: yes/no” but “docs: 0.47” which produces an exact intermediate level of documentation.

10.4 LLM Persona Steering

Prompt engineering techniques like persona steering (“You are a senior Rust developer who writes clean, idiomatic code”) attempt to control LLM output style through natural language instructions [5]. These are probabilistic, non-reproducible, and non-composable. The Style Engine’s coordinate system is deterministic, reproducible, and supports continuous interpolation.

---

11. Future Work

11.1 Style Clustering

Analysis of large codebases could identify natural clusters in the 4D style space — the "styles that programmers actually use" as opposed to the infinite continuum of possible styles. These clusters could inform new named presets beyond the current eight.

11.2 Per-Language Style Adaptation

The current system applies the same style axes across all 37 languages. Language-specific norms (Python's PEP 8, Rust's rustfmt conventions, Go's gofmt standards) could be encoded as per-language bias terms that shift the style space to respect community conventions.

11.3 Temporal Style Evolution

Coding style evolves over time — the C of 1978 differs from the C of 2025. StyleMemory could track temporal trends and suggest styles appropriate to the era of a codebase, enabling new contributions to legacy code to match existing conventions.

11.4 Multi-Objective Optimization

The current loss function uses a fixed weighting of metrics. Pareto-optimal style search across all four metrics simultaneously would enable the discovery of style profiles that cannot be found by any single-objective optimization.

References

[1] D. J. Charlot, "The Pattern Programming Language: Programs as Typed Pattern Graphs," *AGI-Wisdom Technical Report*, February 2026.

[2] K. Munson et al., "Exploring Code Style Transfer with Neural Networks," *arXiv preprint arXiv:2209.06273*, 2022.

[3] Prettier, "Prettier — An Opinionated Code Formatter," *prettier.io*, 2024.

[4] P. S. Langa, "Black: The Uncompromising Code Formatter," *black.readthedocs.io*, 2024.

[5] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *NeurIPS*, 2022.